CSCI 2406 | Computer Organization & Architecture | TOPIC 8

Procedures and the Stack

Procedure calls, stack organization,
parameter passing, and calling conventions

Dr. J. Burns

Topic Outline

1. **Function Calls and Returns**
   - Branch-with-link, the link register, returns, leaf functions, and non-leaf functions.
2. **AAPCS64 Register Rules**
   - Argument registers, return registers, caller-saved registers, and callee-saved registers.
3. **Stack Frames**
   - Stack purpose and lifetime, local-variable storage, allocation, frames, alignment, and entry/exit sequences.
4. **Worked Function Examples**
   - Nested calls, recursion, and common linkage mistakes.

PART 1

Subroutine Linkage & Control Flow

The Link Register (`x30`) and Branch-with-Link (`bl`)

- The **caller** invokes another function, the **callee**, and must be able to resume after it returns.
- **Branch with Link (`bl label`):** Atomically performs two operations:
  1. Saves return address (PC $+ 4$) into the **Link Register (`x30` / `lr`)**.
  2. Branches to target function: PC $\leftarrow$ label.



Direct and Indirect Calls

`bl label` calls a PC-relative target. For a function address held in a register, `blr x2` calls through `x2`; it also writes the return address to `x30`.

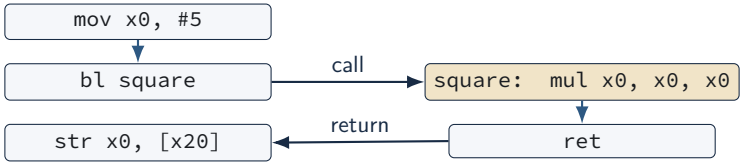
Subroutine Return Mechanics (`ret`)

- **Return from Subroutine (`ret`):** Branches to the address held in a register. With no explicit operand, `ret` uses the link register: PC $\leftarrow x_{30}$.
- A normal `bl` / `ret` pair therefore resumes at the instruction following the original call.

Example: Minimal Leaf Return Sequence

```
caller_func:
  bl helper_func   // x30 = return address; branch to helper
  mov x1, x0       // resumes here after ret
helper_func:
  add x0, x0, #1   // compute result
  ret              // default target register is x30
```

Subroutine Call Execution Trace



Control Flow

`bl square` writes the address of the next instruction into `x30` and branches to `square`. The callee computes the result in `x0`; `ret` resumes at the instruction after `bl`.

Leaf vs. Non-Leaf Functions

Leaf Functions

- Call **no other functions**.
- `x30` is not overwritten by a nested `bl`.
- May still use stack space for local data, but need not save `x30` solely for linkage.

Non-Leaf Functions

- Call one or more **nested subroutines**.
- A nested `bl` overwrites `x30` with the nested return address.
- Preserve the incoming return address before that happens if it will be needed later.

The Non-Leaf Rule

If a function makes another call and must later return to its own caller, its incoming return address must be preserved before `x30` is overwritten.

PART 2

Procedure Call Standard & Register Roles

Why a Calling Convention Is Needed

Separately Compiled Code Must Agree

A program may combine separately compiled C, assembly, and libraries. These components need shared rules: `bl` transfers control and records a return address, but does not define argument locations or register preservation.

- **ABI — Application Binary Interface:** rules that let compiled components work together, including data layout, procedure calls, and object-file conventions.
- **AAPCS64 — Procedure Call Standard for the Arm 64-bit Architecture:** the calling convention within the Arm ABI: where arguments/results go, which registers survive calls, and how the stack is aligned.
- **Example:** for a function adding two 64-bit integers, both sides agree that `x0`/`x1` carry the arguments and `x0` returns the result. Disagreement can corrupt data or execution.

ISA vs. ABI: A64 defines instruction behavior; the ABI defines how compiled components cooperate.

Integer Arguments and Return Values

- **Arguments:** ordinary 64-bit scalar integer and pointer arguments use `x0`–`x7`, subject to AAPCS64 classification rules. Ordinary 32-bit integers use the corresponding `w` registers.
- **Return value:** a 64-bit integer or pointer result normally uses `x0`; a 32-bit integer result uses `w0`.
- **Beyond the registers:** additional arguments assigned to memory use a stack argument area prepared by the caller. The stack is defined in the next section.

Example: Two Arguments, One Result

```
add x0, x0, x1
ret
```

The incoming values in `x0` and `x1` are added; the result returns in `x0`. Floating-point and aggregate classification are outside this example.

Caller-Saved and Callee-Saved Registers

Caller-Saved: `x0`–`x17`

- A call may overwrite these registers.
- The caller must preserve values it still needs after the call.
- `x0`–`x7`: arguments/results; `x8`: indirect result address when required; `x9`–`x17`: temporary uses, including IP0/IP1 in `x16`/`x17`.

Callee-Saved: `x19`–`x29`

- A function that modifies one of these registers must restore its incoming value before returning.
- Values kept in them survive ordinary calls.
- `x29` has the additional conventional role of frame pointer (`fp`).

Example: a loop counter needed after a call can use `x19`, provided the function preserves its caller's original `x19`.

Special-Purpose ABI Registers

- `x29` / `fp`: frame pointer: a stable reference for accessing saved state and local storage on the stack.
- `x30` / `lr`: link register; `bl` and `blr` write the return address here.
- `sp`: stack pointer; marks the active stack boundary and must stay properly aligned.
- `x18`: platform register; do not use it in hand-written examples unless the target platform says it is available.

PART 3

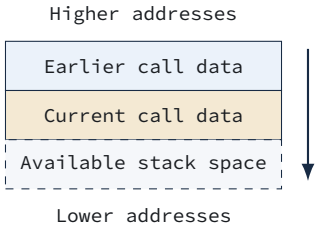
The Runtime Stack & Stack Frames

The Runtime Stack

Definition

The runtime stack is a **per-thread data structure in memory** for temporary function-call state. The OS and runtime establish its memory region. Each thread has its own stack pointer (`sp`).

- Saved return address (`lr`/`x30`) and caller's frame pointer (`fp`/`x29`), when needed.
- Other saved register values.
- Stack arguments, e.g., integer arguments beyond the first eight register arguments.
- Short-lived local variables, arrays, and temporary values that need memory.



Direction: AAPCS64 uses a descending stack: growth is from high to low addresses. This is a convention, not a property of every stack data structure.

Where Does the Compiler Store Local Variables?

Compiler Rule

The compiler chooses storage while preserving observable behavior. A local declaration does not require a stack slot.

- **Register:** keep a value in an available register when no memory address is needed.
- **Stack storage:** typically holds addressable locals, arrays that need memory, and values displaced from registers (**spills**).
- **No storage:** eliminate unused variables, substitute constants, or recompute values.

Taking an address: a local whose address is passed to a function that accesses it needs memory, usually a stack slot, unless optimization removes that need.

Build settings: unoptimized builds commonly use more stack slots; optimization can reduce them.

Stack Lifetime, Balance, and Limits

- **Short-lived storage:** call-related data remains live while needed by the function. Nested callees can allocate more space while the caller's data remains allocated.
- **Balance before return:** release all space allocated by the function and restore required saved registers. sp must equal its entry value; several allocations can be released together.
- **Stack leak:** unreleased allocations that accumulate consume capacity. An incorrect sp at return can also corrupt execution.
- **Finite capacity:** each thread's stack limit depends on the OS, executable, and runtime configuration. Deep recursion or large locals can cause stack overflow.
- **After release:** the space can be reused, so pointers to former locals must not be used. Releasing space does not erase its bytes.

Allocating and Releasing Stack Space

- Stack storage is reserved and released by changing the stack pointer (`sp`).
- **Stack behavior:**
  - Grows **downward** from high memory addresses toward low memory addresses.
  - Stack Pointer (`sp`) points to the current **lowest active address** (top of stack).
  - Follows **Last-In, First-Out (LIFO)** order: the most recently allocated storage is released first.

Allocation vs. Deallocation

- **Allocate:** subtract from `sp`, e.g., `sub sp, sp, #32`.
- Here `#32` means **32 bytes**: room for four 64-bit `x` register slots.
- **Release:** add the same amount back, e.g., `add sp, sp, #32`.

What Is a Stack Frame?

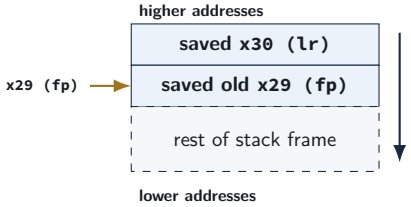
- Each function call may reserve a region of the runtime stack called a **stack frame**.
- A stack frame can hold:
  - an optional **frame record**: the 16-byte pair holding saved old `x29` and saved `x30`,
  - other saved callee-saved registers and space for outgoing stack arguments,
  - local variables, temporary storage, and alignment padding.
- The stack pointer (`sp`) identifies the current stack boundary and changes as stack storage is allocated or released.

Frame Pointer (fp)

`x29` (fp) provides a **stable reference within the frame**, not necessarily its beginning. When a frame record is maintained, fp points to that record. Its address remains fixed while `sp` tracks allocations and releases.

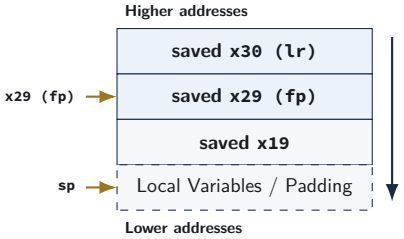
Frame Pointer (x29) and Frame Records

- The **stack frame** is the whole allocation; the **frame record** is a 16-byte part of it.
- The record has two 64-bit fields:
  - `[x29]`: saved previous x29
  - `[x29, #8]`: saved x30
- fp points to the saved old x29 slot. Its stored value points to the caller's frame record, forming a chain.



Example Stack Frame Structure

- A **stack frame** stores the stack-resident state associated with one function invocation.
- In the layout used here:
  1. saved link register (`x30`) and frame pointer (`x29`),
  2. saved callee-saved registers, such as `x19` in the diagram, come below it,
  3. local variables and buffers extend farther downward.
- Arrows identify slots; pointers hold their lowest byte addresses.



AAPCS64 reminder: a callee that modifies any of `x19`–`x29` must preserve its incoming value and restore it before returning. Reading a register alone does not require saving it.

Stack Alignment and Padding Rules

- AAPCS64 requires sp to satisfy 16-byte alignment at public interfaces and when used for stack memory accesses.
- Stack-frame allocation therefore rounds the required stack space up to a multiple of 16 bytes.

Padding is unused space added for alignment. Round the total required bytes up to a multiple of 16: 36 bytes becomes 48 bytes.

Required Space	Stack Frame Size	Example Allocation
16B saved fp/LR	16 bytes	`stp x29, x30, [sp, #-16]!`
16B saved fp/LR + 20B locals	48 bytes	allocate a 48-byte stack frame and place saved state/locals within it
16B saved fp/LR + 32B locals	48 bytes	allocate a 48-byte stack frame

Function Prologue Mechanics

- A **prologue** allocates stack storage and saves required state on function entry.
- Here the `x29`/`x30` frame record stays at the high-address end of this function's stack frame.
- Saving one 8-byte `x` register still reserves 16 bytes so `sp` remains aligned.

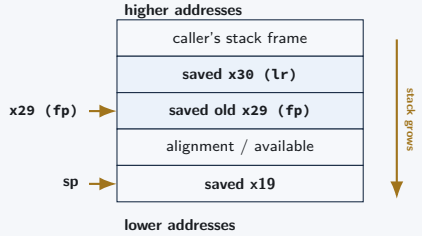
Prologue Template

```
my_func:
    // save old fp and LR as frame record
    stp    x29, x30, [sp, #-16]!

    // fp = address of saved old x29
    mov    x29, sp

    // save x19 below the frame record
    str    x19, [sp, #-16]!
```

Stack Layout



Function Epilogue Mechanics

- An **epilogue** restores saved state and releases stack storage before return.
- Restore any saved callee registers before restoring the frame record and returning.

Epilogue Template

```
// 1. Restore saved callee register; release its slot
ldr    x19, [sp], #16

// 2. Restore original fp and LR; release frame record
ldp    x29, x30, [sp], #16

// 3. Return to caller
ret
```

`ldp` restores both the caller's fp and the saved LR; `ret` then branches to LR (`x30`). Post-indexing releases each allocation after loading it.

PART 4

Worked Assembly Examples & Patterns

Worked Example 1: Non-Leaf Nested Function

C Code: `int64_t compute(int64_t x)`

```
return add_three(x, 10, 20);
```

AArch64 Assembly Implementation

```
// Input: x0 = x
compute:
          stp x29, x30, [sp, #-16]!   // save fp and LR
          mov x29, sp                 // establish frame record
          mov x1, #10                 // 2nd argument
          mov x2, #20                 // 3rd argument
          bl add_three                // nested call overwrites x30
          ldp x29, x30, [sp], #16     // restore fp/LR
          ret                         // return to caller
```

Worked Example 2: Recursive Factorial

C Logic

```
uint64_t factorial(uint64_t n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

Each invocation preserves its own return address and the current value of n across the recursive call.

AArch64 Assembly

```
factorial:
    cmp  x0, #1
    b.ls .Lbase

    stp  x29, x30, [sp, #-32]!
    mov  x29, sp
    str  x19, [sp, #16]
    mov  x19, x0
    sub  x0, x0, #1
    bl   factorial
    mul  x0, x0, x19
    ldr  x19, [sp, #16]
    ldp  x29, x30, [sp], #32
    ret

.Lbase:
    mov  x0, #1
    ret
```

Common Function Linkage Bugs in Assembly

- **1. Overwriting the Return Address:** A nested `bl` overwrites `x30`; failing to preserve the incoming return address can return control to the wrong location.
- **2. Corrupting Callee-Saved Registers:** Modifying `x19`–`x28` without restoring them violates the calling convention.
- **3. Stack Misalignment:** Violating AAPCS64 stack-alignment rules breaks ABI assumptions and can fault where SP-alignment checking is enabled.
- **4. Unbalanced Stack Pointer:** Failing to restore `sp` correctly can corrupt stack-resident state and later restores.

SUMMARY & ASSESSMENT

Review and Self-Test Questions

Summary

- **Linkage Primitives:** `bl`/`blr` write the return address to `x30`; `ret` normally returns through `x30`.
- **AAPCS64 Register Contract:**
  - Scalar arguments commonly begin in `x0`–`x7`; most scalar results return in `x0`.
  - `x0`–`x17` are caller-saved; `x19`–`x28` are callee-saved.
- **Stack Frames:** A stack frame holds saved linkage state, preserved registers, and local storage while maintaining required alignment.
- **Prologue/Epilogue:** Paired `stp`/`ldp` instructions commonly save and restore `x29`/`x30` while updating `sp`.

Self-Test Questions: Linkage & AAPCS64

1. **Leaf vs. Non-Leaf Function Linkage:** How do leaf and non-leaf functions differ regarding link-register management and return-address preservation?
2. **Register Preservation Contracts:** Suppose function `foo()` calls `bar()`. If `foo()` needs to maintain a loop counter value across the call, should it store that counter in `x9` or `x19`? Why?
3. **Parameter Passing Limits:** How are integer parameters handled when calling a C function declared with 10 scalar arguments: `void test(int a1, ..., int a10)`?

Self-Test Questions: Stack Frame Management

1. **Stack Allocation Alignment:** A non-leaf function saves `x29`/`x30` (16 bytes) and needs 20 bytes of local buffer space. What is the minimum stack-frame size it must allocate to maintain AAPCS64 compliance?
2. **Prologue/Epilogue Verification:** Identify the bug in the following epilogue:

```
ldp x29, x30, [sp], #16
ldr x19, [sp, #16]
ret
```

3. **Frame Pointer Chains:** Why need fp not point to the beginning of the whole frame? What do `[x29]` and `[x29, #8]` contain, and how can a debugger follow the chain?